

uv



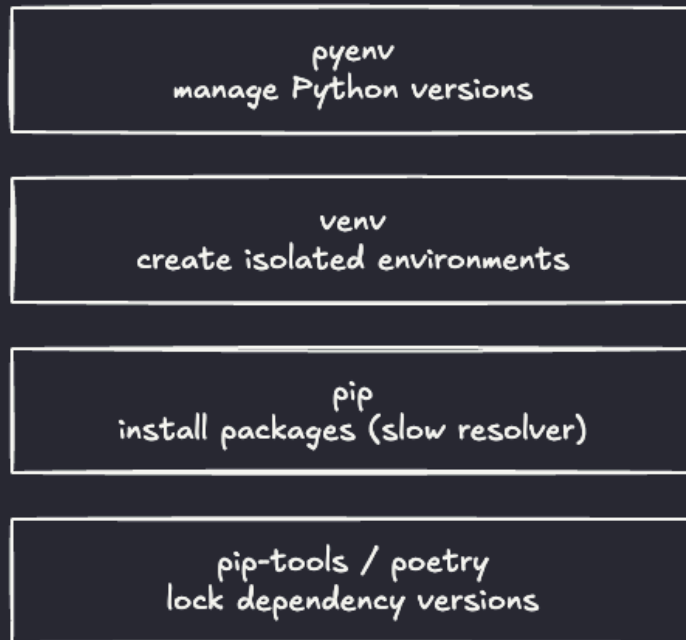
SophiArch

The Problem: Too Many Tools

- **pyenv** to manage Python versions
- **venv** to create isolated environments
- **pip** to install packages — with a notoriously slow dependency resolver
- **pip-tools** or **poetry** on top, just to get reliable version locking
- Four tools, four config formats, four things that can go out of sync

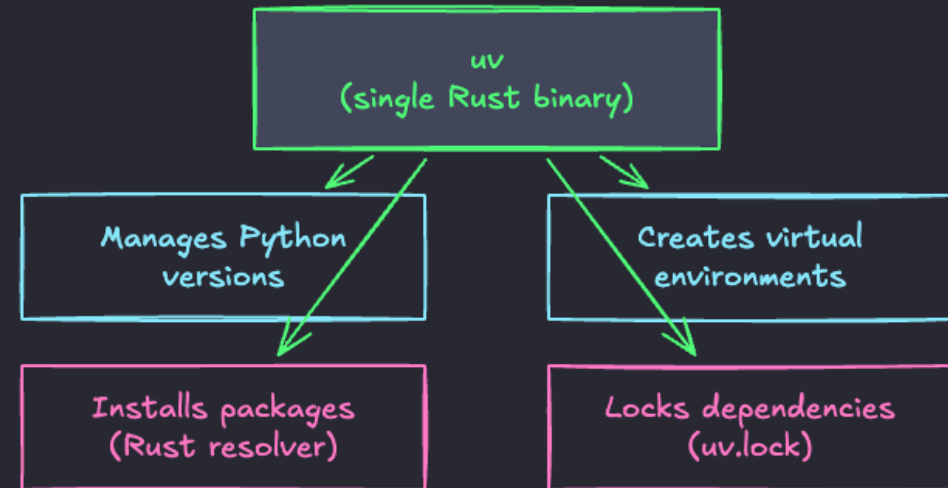
Fragmented Toolchain vs. uv

Fragmented Toolchain



4 separate tools, 4 sets of config, 4 failure modes

One Tool: uv

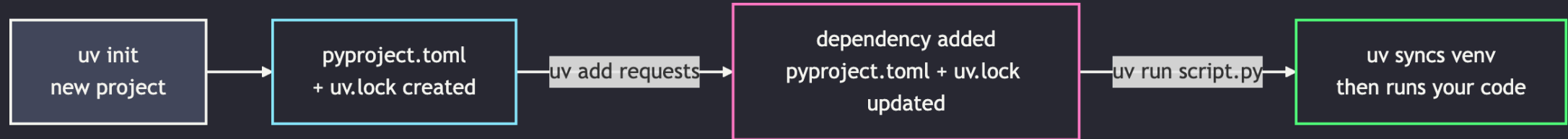


1 tool, written in Rust — 10-100x faster installs

What `uv` Actually Is

- A single, fast, all-in-one Python package and project manager, written in Rust
- Built by **Astral** — same team behind the `ruff` linter/formatter
- Drop-in replacement for `pip`, `venv`, `pip-tools`, `virtualenv`, and (largely) `pyenv`
- Manages `pyproject.toml` and `uv.lock` automatically — no manual `requirements.txt` bookkeeping

How the Workflow Flows



Installation

```
# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows (PowerShell)
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"

# Or via pip, if you already have Python installed
pip install uv
```

- Installs a single self-contained binary — no Python required to bootstrap it
- Verify with `uv --version`

Demo: Start a New Project

```
# 1. Scaffold a new project
```

```
uv init my-project
```

```
cd my-project
```

```
# 2. Add a dependency – creates the venv automatically if needed
```

```
uv add requests
```

```
# 3. Run code inside the managed environment – no activate step
```

```
uv run python -c "import requests; print(requests.__version__)"
```

Demo: Everyday Commands

```
# Add a dev-only dependency (e.g. a test runner)
```

```
uv add --dev pytest
```

```
# Remove a dependency
```

```
uv remove requests
```

```
# Recreate the exact environment from uv.lock (e.g. on a teammate's machine)
```

```
uv sync
```

```
# Run a script or command inside the project's environment
```

```
uv run pytest
```


uv vs. Traditional Commands

Traditional	uv equivalent
<code>python -m venv .venv</code>	<i>(automatic — created on first <code>uv add</code> / <code>uv run</code>)</i>
<code>source .venv/bin/activate</code>	<i>(not needed — use <code>uv run ...</code>)</i>
<code>pip install requests</code>	<code>uv add requests</code>
<code>pip freeze > requirements.txt</code>	<i>(automatic — <code>uv.lock</code> stays in sync)</i>
<code>pip install -r requirements.txt</code>	<code>uv sync</code>
<code>pyenv install 3.12</code>	<code>uv python install 3.12</code>

Common Pitfalls

- **Editing** `pyproject.toml` **by hand for dependencies** — use `uv add` / `uv remove` so `uv.lock` stays in sync
- **Forgetting** `uv run` — running `python script.py` directly uses whatever Python is on your PATH, not the project's venv
- **Committing** `.venv/` **but not** `uv.lock` — it's the reverse of what should be committed; `.venv/` is disposable, `uv.lock` is the source of truth
- **Mixing** `pip install` **into a uv-managed project** — installs outside `uv.lock` tracking and can drift from what teammates have

Cheat Sheet

Action	Command
Install uv	<code>curl -LsSf https://astral.sh/uv/install.sh sh</code>
New project	<code>uv init <name></code>
Add a dependency	<code>uv add <package></code>
Add a dev dependency	<code>uv add --dev <package></code>
Remove a dependency	<code>uv remove <package></code>
Sync environment to lock file	<code>uv sync</code>
Run code in the project env	<code>uv run <command></code>
Install a Python version	<code>uv python install <version></code>

References

Topic	Source
Official  documentation	Astral. (2026). uv — An extremely fast Python package manager . Astral Docs.
 GitHub repository	Astral. (2026). astral-sh/uv . GitHub.
 benchmark results	Astral. (2026). uv: Benchmarks . GitHub.

Recap & Practice

- `uv` replaces `pyenv` + `venv` + `pip` + `pip-tools/poetry` with one fast, unified tool
- `uv add`, `uv run`, and `uv sync` cover nearly everything you'll need day to day
- **Before your next assignment:** `uv init` the project instead of `python -m venv`